

Simulation Experiments in Econometrics

Rasmus Jørgensen

February 2015

1 Introduction

Simulation experiments are a useful and powerful method for investigating the properties of econometric estimators and test statistics. The power of simulation experiments derive from being able to define and control the statistical environment in which the investigator specifies the data generating process (DGP) and generates data used in subsequent experiments.

Simulation experiments in Econometrics are often referred to as Monte Carlo experiments. No one seems to know for sure how the term originated, but it probably has something to do with the famous casino, symbolizing the laws of chance in a highly controlled environment.

Monte Carlo studies can be used to verify well-known properties such as unbiasedness of the OLS estimator under assumptions MLR.1–MLR.4. More interestingly, simulation experiments can also be used to investigate how the OLS estimator is biased when we deviate from the basic MLR assumptions.

As these examples illustrate, Monte Carlo experiments can be used to verify statistical properties that have been established analytically. Econometricians, however, often use estimators that have well-known asymptotic properties, but for which nothing can be said analytically in finite samples.

E.g., it has been established that the OLS estimator is asymptotically normally distributed under MLR.1–MLR.5. It is, however, unclear if this result of asymptotic normality provides a good approximation in finite samples. That is, how *normal* can we expect the distribution of the OLS estimator to be in small samples? The answer to this question is theoretically ambiguous, but it is nonetheless important if asymptotic test statistics are to be used. In this case, simulation experiments are helpful tools to investigate if the asymptotic properties are good approximations or not.

This note proceeds by explaining every step of a simulation experiment and providing detailed instructions for how to implement an experiment in STATA.

2 Monte Carlo Experiments

The basic steps in any Monte Carlo experiment are:

1. Simulate data on a computer
2. Calculate relevant estimates or statistical measures for each generated sample in 1.
3. Repeat 1 and 2 many times and save the estimates in each iteration.
4. Provide summary statistics of all simulated estimates and statistical measures.

A Monte Carlo experiment is based on simulations of a specific model. To perform the simulation, it is necessary to specify the data generating process (DGP) in the first step. A DGP can be illustrated by the following example:

Example: DGP for simple linear regression model

$$y_i = \beta_0 + \beta_1 x_i + u_i, \tag{1}$$

$$\beta_0 = 1, \beta_1 = 2, x \sim N(2, 4), u \sim N(0, 1), n = 100 \tag{2}$$

In this example, we have chosen specific values for β_0 and β_1 and we have specified the distributions of the stochastic variables, x and u . The main challenge of simulating data from this DGP is to simulate draws—that is, observations—from the distributions of x and u . Fortunately, it is easy to generate random numbers on a computer from distributions such as the normal and the uniform. This note will show you how to do this in STATA in the next section.

Assuming we have drawn 100 observations on x and u from the relevant distributions, we can next calculate 100 observations for y using the assumptions in (1) and (2). By specifying the DGP, we can define and control the statistical environment and are now able to generate artificial samples with information on x and y . This concludes the first step of our Monte Carlo experiment.

In the second step, we calculate the objects of interest. Continuing with the example from before, we estimate the parameters of the simple regression model using data on x and y from step 1 and the OLS estimator. In some sense, we will temporarily forget how the DGP has been specified and simply calculate the relevant estimates using the data at hand.

In step 3, we continue our experiment by repeating steps 1 and 2 over and over again. By simulating the objects of interest 10,000 times, then it is possible to obtain the distribution of the OLS estimates across all samples. As such, a Monte Carlo experiment is analogous to the thought experiment underlying both statistics and econometrics, where we derive the theoretical properties of, *e.g.*, the OLS estimator across hypothetical samples. In our Monte

Carlo experiment, we are replacing the hypothetical samples with specific, but generated samples, allowing us to obtain, *e.g.*, the distribution of the OLS estimator.

Generally, it is desirable to have as many replications as possible because it increases the precision with which the distribution of the estimates can be determined. The cost of using a very high number of replications is that it can take a long time to carry out the experiment—although this, of course, depends on the computer. In practice, 10,000 replications are often used.

Once steps 1-3 have been carried out, it is possible to investigate the empirical distribution of the estimator or test statistics. E.g., it is well-known that OLS is the best linear and unbiased estimator under MLR.1–MLR.5. This implies that the mean of the 10,000 OLS estimates from the experiment should be equal to the true value of the parameters and that they are estimated with the smallest possible variance when compared to all other linear unbiased estimators. In step 4, we thus provide a descriptive analysis of the estimates obtained from the simulation experiment.

This concludes the four fundamental steps of designing and carrying out Monte Carlo experiments.

At this point, you may be asking what can we *really* learn from Monte Carlo experiments? It is true that all simulated results depend on the assumptions regarding parameter values and statistical distributions when specifying the DGP. But it is *exactly* for this reason that simulation studies can never prove a theoretical property such as unbiasedness.

Monte Carlo experiments can, however, be used to illustrate certain statistical properties or at least provide insights when analytical results are unavailable. In the latter case, there is in some ways a trade-off between the calculation costs of humans versus computers. And it is usually a lot faster to carry out a simulation study than to come up with an analytical proof which may not even exist.

In practice, econometricians use simulation studies to investigate how estimators behave in different environments. E.g., what is the variance of the OLS estimator in situations with *uncorrelated* regressors versus *correlated* regressors? In general, Monte Carlo experiments are not very informative on their own, but comparisons of either different estimators in the same environment or the same estimator in different environments are usually much more informative.

Having outlined the fundamental principles of simulation experiments, we will now turn to the practical implementation in STATA. We begin with generating random numbers on the computer in the next section.

3 Generating pseudorandom numbers in STATA

Computers cannot generate *truly* random numbers. Computer scientists have, however, developed sophisticated, but deterministic, algorithms that generate numbers that are approximately random. These pseudorandom number generators usually require some initial value in order for them to generate draws from a given distribution. This is the so-called *seed* number.

By using the same seed number, it is always possible to generate the same sequence of pseudorandom numbers which is useful if you want to recreate the same simulation experiment again.

In STATA, random draws from a uniform distribution can be generated using **runiform()**. In the example below, we draw 1000 observations from the uniform distribution in STATA:

```
. clear all
. set obs 1000
obs was 0, now 1000

. set seed 117
. generate u=runiform()

. summarize
```

Variable	Obs	Mean	Std. Dev.	Min	Max
u	1000	.4982696	.2825431	.001939	.9990226

Similarly, it is possible to generate draw random numbers from $N(0, 1)$ using **rnormal()**. If we instead want to draw from $N(\mu, \sigma^2)$, we can define $z = \mu + \sigma x$, where $x \sim N(0, 1)$. In the example below, $z \sim N(2, 9)$:

```
. generate x=rnormal()

. generate z=2+3*x

. summarize x z
```

Variable	Obs	Mean	Std. Dev.	Min	Max
x	1000	.0194852	.9932642	-3.660761	2.831711
z	1000	2.058455	2.979793	-8.982283	10.49513

We now have the tools to generate one artificial sample of y and x . Below we simulate one sample from the following DGP:

$$y_i = \beta_0 + \beta_1 x_i + u_i, \quad (3)$$

$$\beta_0 = 1, \beta_1 = 0.25, x \sim N(0, 1), u \sim U(-1, 1), n = 50 \quad (4)$$

```
. clear all
. set seed 117
. set obs 50
obs was 0, now 50

. generate u=2*(runiform()-0.5)
. generate x=rnormal()
. generate y=1+0.25*x+u

. summarize y x
```

Variable	Obs	Mean	Std. Dev.	Min	Max
y	50	.9946031	.5889837	-.2247419	2.723315
x	50	.0342411	1.002593	-1.831092	2.930734

Regressing y on x , we obtain a slope estimate of $\hat{\beta}_1 = 0.20$ for this particular sample which is different, but not very far from the true value $\beta_1 = 0.25$.

```
. regress y x
```

Source	SS	df	MS	Number of obs =	50
Model	2.02109895	1	2.02109895	F(1, 48) =	6.48
Residual	14.9770868	48	.312022642	Prob > F =	0.0142
Total	16.9981858	49	.34690175	R-squared =	0.1189
				Adj R-squared =	0.1005
				Root MSE =	.55859

y	Coef.	Std. Err.	t	P> t	[95% Conf. Interval]
x	.2025681	.0795922	2.55	0.014	.0425373 .362599
_cons	.9876669	.0790435	12.50	0.000	.8287393 1.146595

Now we know how to simulate data from a fully specified DGP and obtain estimates using, *e.g.*, **regress**. In the next section, we will show how to repeat this procedure 10,000 times.

4 Simulation study of the OLS estimator

Monte Carlo experiment can be implemented in STATA using different commands. We will introduce two approaches in this section. First, we will run a simulation experiment using the **simulate** command. This is STATA's built-in command for Monte Carlo experiments. It is fairly easy to use and it works well with STATA's other commands such as **regress**.

The second approach is based on STATA's matrix programming language **mata**. This programming language is very basic in the sense that **mata** will only compute the matrix operations we ask it to. On the other hand, **mata** offers full customization and is significantly faster than **simulate**.

4.1 Simulate

In order to use **simulate**, we first need to define the DGP in a *program*. You can think of a program as a series of commands that will be executed every time the program is activated. To implement the DGP from before (see (3) and (4)), we define the program *olsdata* below. In the program, we instruct STATA to draw 50 observations of x and u from the relevant distributions and then calculate y with our assumptions for β_0 and β_1 . Finally, we run a regression of y on x and save the estimated slope in a scalar named *b1*.

```
program olsdata, rclass
    drop _all

    *SET NUMBER OF CURRENT OBSERVATIONS
    set obs 50

    *DATA GENERATING PROCESS
    generate x = rnormal()
    generate u = 2*(runiform()-0.5)
    generate y = 1+0.25*x+u

    *CALCULATE OLS ESTIMATES AND SAVE OLS SLOPE IN B1
    regress y x
    return scalar b1=_b[x]
end
```

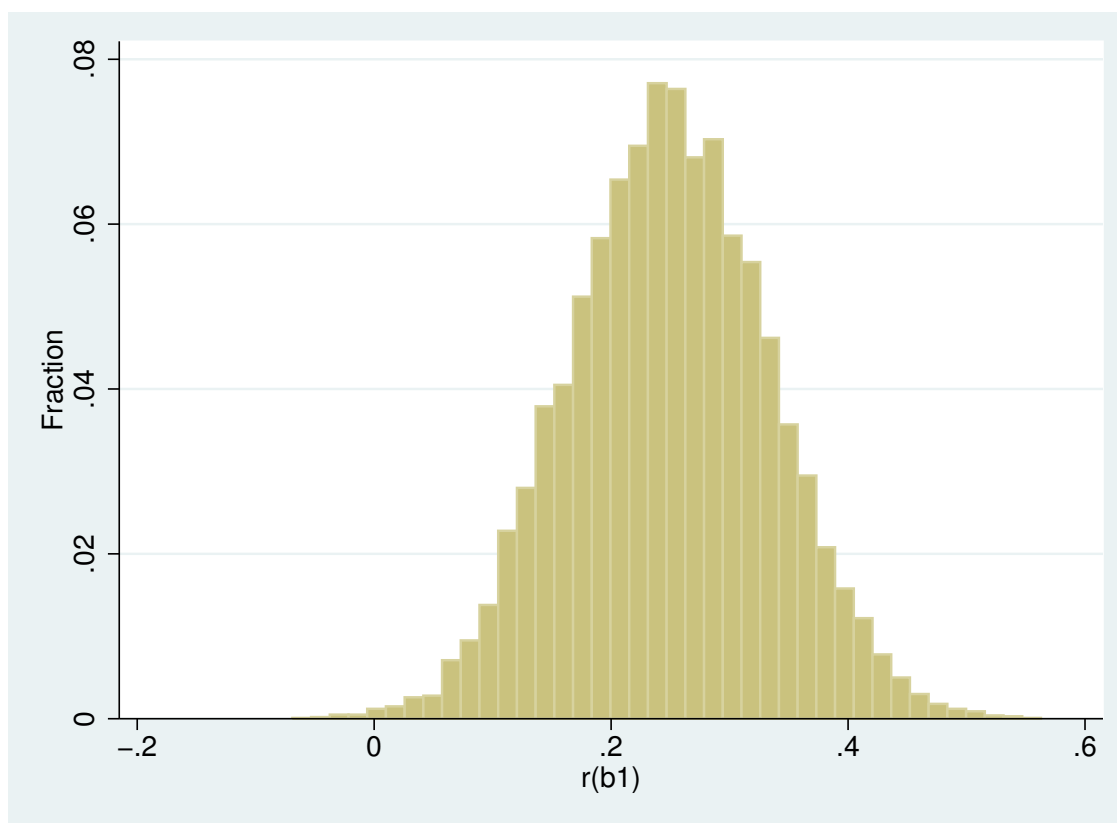
Next, we run our Monte Carlo experiment using the **simulate** command. Specifically, we tell **simulate** to run our program *olsdata* 10,000 times using the seed number 117. Moreover, the estimates in *b1* from each iteration is stored in the variable *betahat*.

```
simulate betahat=r(b1), seed(117) reps(10000) nodots:olsdata
```

Upon completion, **simulate** will create a data file containing all 10,000 slope estimates. Using **summarize**, we can see that the average of those estimates is equal to 0.25. This is to be expected as the specified DGP satisfies MLR.1–MLR.4, and as a result, the OLS estimator is unbiased.

. summarize					
Variable	Obs	Mean	Std. Dev.	Min	Max
-----+-----					
betahat	10000	.2491458	.0839517	-.0690546	.5627797

We can also draw up a histogram for the 10,000 estimates of β_1 . In the figure below, we see that the distribution of the OLS estimator is approximately normal, despite the fact that the errors are assumed to be uniformly distributed in the DGP. This finding illustrates the asymptotic normality of OLS under MLR.1–MLR.5.



4.2 MATA

We will now show to run the same Monte Carlo experiment using the programming language **mata**. As mentioned earlier, **mata** is a basic matrix programming language that is very fast and very versatile.

We begin by defining a few local variables for the number of observations and the number of replications in our experiment. This is convenient as we can refer to these local variables later on (making it easier to keep track of the appropriate matrix dimensions).

```
clear all
local nobs 50
local nrep 10000
```

We will now call **mata**. Note that once you submit **mata**, STATA enters the matrix programming module and only **mata**-commands can be used. To exit **mata**, you need to submit *end*.

The first object defined in **mata** is the matrix *R*. This matrix has dimension (10,000×1) and will be used to store the 10,000 estimates of β_1 . Next, we define the matrix *cons* as a (50×1) vector with all elements equal to 1. Then, we use a for-loop to carry out the same set of instructions 10,000 times. Note that the loop ends with *}*. The way to think about the for-loop is that it starts with replication 1 and ends with replication 10,000.

(continues on next page)

```
mata                                     /* start mata */
R=J('nrep',1,0)                        /* matrix to store results */
cons=J('nobs',1,1)                    /* vector with all 1s (constant) */
rseed(117)                             /* seed number */
for (i=1; i<='nrep'; i++) {           /* loop over replications */
    u=2*(runiform('nobs',1)-0.5*cons)  /* draw uniform errors */
    x=rnormal('nobs',1,0,1)           /* draw normal expl. var */
    y=1*cons+0.25*x+u                 /* calculate y */
    X=(x,cons)                        /* define big x matrix */
    beta_ols=(invsym(X'*X))*(X'*y)    /* calculate OLS estimates */
    R[i]=beta_ols[1,1]                /* save OLS estimate */
}

st_addobs('nrep')                      /* save mata matrix as state data file */
st_addvar("float", "OLS_beta")
st_store(., "OLS_beta", R)
end
```

The part of the code that is inside the the for-loop curly brackets contains the instructions that will be executed in every iteration. As before, u is a (50×1) vector of draws from a $U(-1, 1)$. The vector x contains similar draws from a standard normal distribution. Note that in **mata** `rnormal()` has 4 inputs. Inputs 1 and 2 specify the $(r \times c)$ -matrix you want to draw, while inputs 3 and 4 specify the mean and the variance of the normal distribution. As before, we can calculate y from our fully specified DGP.

In **mata**, we have to calculate $\hat{\beta} = (X'X)^{-1}X'y$ by hand. We can do this by first concatenating the vectors containing the intercept and the explanatory variables. With X and y defined, we can simply calculate using the OLS estimator from before. At the end of each iteration, we save the estimated slope parameter in R before moving on to the next replication.

Upon execution of all 10,000 replications, we convert the **mata** matrix R to a STATA data file. Using **summarize**, we calculate the average of those 10,000 slope estimates to 0.25. Once again, we have verified the unbiasedness of OLS given the assumptions underlying our Monte Carlo experiment (which satisfy MLR.1–MLR.4).

. summarize					
Variable	Obs	Mean	Std. Dev.	Min	Max
-----+-----					
OLS_beta	10000	.250245	.0853164	-.0710048	.5649038

5 Summary

This note has introduced the fundamental concepts of Monte Carlo experiments. Simulation experiments serve a number of purposes, from verifying well-known properties that have been established analytically to replacing analytical results when they are difficult or impossible to obtain.

The note has also presented a template for creating your own simulation experiments in STATA using either the **simulate** command or the **mata** programming language. The do-file used in this note is available on the course website.